The Uffizi Guide

A plain-language tour of the algorithms:
how they work, how they differ, and why they produce the numbers they do

Uffizi RL project

Abstract

This guide assumes you know nothing about reinforcement learning. It starts from the everyday idea of learning by trial and error, builds up the vocabulary one piece at a time, and only then writes down an equation, explaining what every symbol is doing. It covers all the methods the project uses: tabular Q-learning and Double Q-learning on the toy museum; the handcrafted rules we compare against; MaskablePPO, plain PPO, and DQN on the full museum; and the training and evaluation scaffolding (action masking, the curriculum, common random numbers, multiple seeds). The last third of the guide does something most write-ups skip: it explains, method by method, *why* the numbers came out the way they did, so that you can look at our tables and figures and say not just *what* happened but *why* it had to happen.

Contents

1 The problem, in one breath

The Uffizi Gallery in Florence takes roughly five thousand visitors a day through about a hundred rooms. Almost all of them want the same four things: the two Botticelli paintings, the Leonardo, and the Raphael. So those four rooms are jammed shoulder to shoulder while the rest of the museum sits nearly empty. That is the whole problem in a sentence: not too many people for the building, but too many people for four rooms at the same moments.

The project attacks this on two fronts that, for most of the document, you should keep firmly apart.

The first front is the *museum-wide simulation*. We build an artificial crowd of five thousand people, send them through the museum minute by minute, and ask what happens to congestion and to visitor satisfaction if the management changes the rules (timed entry, booked slots for the masterpieces, longer hours, and so on). That front does not use any of the learning algorithms in this guide; it is a simulation of many simple people, not one clever one.

The second front, and the one this guide is about, is the *single optimal visitor*. Take the redesigned museum as fixed, and ask: what should one smart visitor actually *do*? When should they book? Which masterpieces? How should they pace the day so they arrive at each painting at the right time and still get out before closing? That is a sequence of decisions made under uncertainty, with consequences that only show up later. That kind of problem is exactly what reinforcement learning is for, and it is why the algorithms below exist in the project at all.

How the two fronts meet, and why we keep them apart. The two fronts connect at a single gate. The simulation decides whether a set of management changes is even acceptable: a director will adopt a redesign only if it raises visitor welfare *without* collapsing ticket revenue, so the simulation's job is to find a bundle of changes (booked masterpiece slots, longer hours, gentle pricing, and so on) that passes that test. Only once a redesign passes does the second front open: take that redesigned museum as fixed, and ask what a single optimal visitor does inside it. So the learning algorithms in this guide never touch the question "is the redesign worth doing?"; they answer the sharper question "given that we did it, how should a smart visitor use it?" Keeping the two apart is what stops us from confusing an *average* over a whole messy population (what the simulation reports) with the *best* behaviour of one ideal visitor (what the agent learns). Those are different questions with different answers, and much of the confusion in this kind of project comes from running them together.

2 The vocabulary of reinforcement learning

Before any algorithm, we need six words. We will introduce them with the museum visitor as the running example, because every one of these words has a concrete meaning here.

State. The *state* is everything the decision-maker knows at the moment it has to act. For our visitor, a state might be: "I am in room A11, it is 11:40, this room is crowded, and I have already seen Botticelli and Leonardo." The state is a snapshot. Crucially, a good state contains *everything that matters for the next decision* and nothing the decision cannot depend on. We will see later that getting the state wrong, leaving out something the right action secretly depends on, quietly breaks the learning.

Action. The *action* is the choice available in a state: "move to the next room," "stay another minute," or in the booking problem, "reserve the Leonardo slot for 35 days from now." The set of

legal actions can depend on the state (you can only walk to a room that is physically next door), and that simple fact will turn out to matter enormously.

Reward. The *reward* is a single number the world hands back after each action, measuring how good that step was right now. In the museum it is built from the art you appreciated this minute, discounted if the room was crowded, minus a small cost for time spent. Reward is local and immediate. It is *not* the same as success; success is about the whole visit, and the reward only scores one step.

Return. The *return* is the total reward summed over the entire visit, from now until you walk out the door. This is what we actually care about. The whole difficulty of the field is that you choose actions one at a time, getting only immediate rewards, but you are graded on the return, which depends on every future step too.

Discount. Because the future is uncertain and a payoff now is worth a little more than the same payoff later, we add up the rewards with a *discount factor* γ between 0 and 1. A reward k steps in the future is multiplied by γ^k. The discounted return from time t is

$$G_t \;=\; r_t \;+\; \gamma\, r_{t+1} \;+\; \gamma^2 r_{t+2} \;+\; \cdots \;=\; \sum_{k=0}^{\infty} \gamma^k\, r_{t+k}. \tag{1}$$

Read equation (1) slowly. G_t is the return starting now. r_t is the reward we get this step, counted in full. The next reward r_{t+1} is multiplied by γ, the one after by γ^2, and so on, so rewards far in the future are shrunk toward zero. If γ is close to 1 (we use 0.99 and higher), the agent is *far-sighted*: a reward forty steps away still counts for a lot, which is essential here because the payoff for a good booking can land hundreds of minutes later. If γ were small, the agent would be a short-term creature that ignores the future.

A worked example of the return. Numbers make this concrete. Suppose a short visit hands out rewards $10, 0, 6, 4$ over four minutes and then ends. The plain (undiscounted) total is 20. With $\gamma = 0.99$ the return is $10 + 0.99 \cdot 0 + 0.99^2 \cdot 6 + 0.99^3 \cdot 4 = 10 + 0 + 5.88 + 3.88 = 19.76$, only a hair under 20 because a discount near one barely shrinks anything. Now redo the sum with a short-sighted $\gamma = 0.5$: $10 + 0 + 0.25 \cdot 6 + 0.125 \cdot 4 = 12.0$. The same visit is "worth" 19.76 to a patient agent and only 12.0 to an impatient one, because the impatient one almost throws away the later 6 and 4. This is the entire reason we set γ as high as 0.999 for the booking agent: the reward for booking well does not arrive until check-in, possibly four hundred minutes later, and at $\gamma = 0.999$ a reward four hundred steps out is still worth $0.999^{400} \approx 0.67$ of its face value, whereas at $\gamma = 0.99$ it would be worth $0.99^{400} \approx 0.018$, effectively invisible. The discount is not a technicality; it decides whether the agent can even feel the consequence of its most important choice.

Policy and value. A *policy*, written π, is the agent's strategy: a rule that, given a state, says which action to take (or gives a probability to each action). Learning is nothing more than improving the policy. To improve it we need a way to score situations, and that is the *value*. The *action-value* $Q(s, a)$ is the return we expect if we take action a in state s and then act well forever after:

$$Q^{\pi}(s, a) \;=\; \mathbb{E}[\, G_t \mid s_t = s,\; a_t = a,\; \text{then follow } \pi \,]\,. \tag{2}$$

The symbol $\mathbb{E}[\cdot]$ means "the average over all the random things that could happen." So $Q(s, a)$ answers a very practical question: "if I do a here and play well afterward, how much total reward

should I expect?" If we knew Q exactly, the best policy would be trivial: in every state, take the action with the largest Q. Every algorithm in this guide is, at heart, a different way of getting at Q or at the policy directly.

The Markov decision process. Put these together and you have a *Markov decision process* (MDP): states, actions, a reward for each step, a rule for how the state changes, and a discount. "Markov" just means the state is a complete enough summary that the future depends only on the present state and action, not on the whole history of how you got there. When we later say a model "broke because the task was not Markov in the state it could see," we mean exactly this property was violated: the right action depended on something missing from the state.

One step of the visitor's life, start to finish. It helps to watch a single turn of the loop, because every algorithm in this guide is just a different way of improving one piece of it. The visitor is in a state s (room A11, 11:40, crowded, Botticelli already seen). The policy looks at s and chooses an action a (say, move on to the corridor). The world responds: it hands back a reward r for that step (a little appreciation banked, discounted because the room was crowded, minus the time cost) and moves the visitor to a new state s' (in the corridor, 11:41). Now the loop repeats from s'. The visit is a long chain of these (s, a, r, s') transitions, and the agent's only job is to make choices so that the *sum* of all those little rewards, the return, is as large as possible. Value-based methods (Q-learning, DQN) attack this by learning the number $Q(s, a)$ for each move and then always picking the largest; policy-gradient methods (PPO) attack it by directly nudging the probabilities of the moves up or down. Keep this one-step picture in your head; everything below is a variation on it.

3 The five elements, made concrete for each algorithm

A natural question, having met the six words, is: "so which exactly is the state in Q-learning? And in PPO?" The clean answer carries an important idea: *state, action, reward, return, and discount belong to the problem, the MDP, not to the algorithm.* An algorithm is a recipe for learning a good policy *within* a given MDP; it does not change what the state or the reward is. So the right way to read the question is by problem, and the project has exactly two. The first is the *toy museum*, solved by both tabular Q-learning and Double Q-learning. The second is the *booking task* on the full museum, solved by DQN, PPO, and MaskablePPO. Within each problem the five elements are identical across the algorithms that share it; what changes is only how the state is *represented* and how the action is *chosen*.

3.1 The toy MDP (Q-learning and Double Q-learning)

These two algorithms face the *same* MDP; nothing in this list differs between them.

State
A discrete five-part label: (current room; time-slice of the day, one of ten; Botticelli crowd level, one of four; Leonardo crowd level, one of four; a bitmask of which masterpieces have been seen). About 123,000 distinct states, stored in a table.

Action
From the current room: *stay*, or *move* to one of the physically adjacent rooms. Only legal moves are offered.

Reward
Per step, importance $\times$ novelty$/(1 + \alpha_A \,\text{density}^2) -$ step cost, with $\alpha_A = 6$ and step cost 0.5, plus a closing-time pressure, an exit bonus (+15), and a no-exit penalty (-50).

Return
The discounted sum of those per-step rewards over the whole visit, $G_t = \sum_k \gamma^k r_{t+k}$ (equation (1)).

Discount
$\gamma = 0.99$.

What differs between the two algorithms is only the bookkeeping. Q-learning stores one table $Q(s, a)$ and updates it with the single-table rule (equation (3)), choosing the action with the largest entry in the row. Double-Q stores *two* tables and updates them with the decorrelated rule (equation (5)), choosing by the average of the two. Same state, same actions, same reward, same return, same discount.

3.2 The booking MDP (DQN, PPO, and MaskablePPO)

These three (the intervened arms of the algorithm comparison) also face the *same* MDP.

State
A vector of about 25 numbers: a booking-phase flag and step; the lead time chosen so far; how many slots are still open at each of the four lead options; the time of day; the three booked windows currently held; the three check-ins already made; a small "access read" of the current room (is it a masterpiece, is it unbooked, is it open now, how long until it opens); the three frozen arrival-time estimates; and the current room's remaining value, importance, crowd density, and egress slack.

Action
One of four (`Discrete(4)`). In the booking phase it sets the lead time, one of $\{1, 7, 21, 35\}$ days, and then, per masterpiece, whether to take the offered slot; in the visit phase it is *stay* or *move on*.

Reward
The per-minute museum reward (appreciation under the crowd discount, closing pressure, a completion bonus, the check-in bonus and waiting penalty, a small step cost), plus the booking-phase terms (the immediate off-pace penalty and the no-show penalty).

Return
The discounted sum of those rewards over the two-phase episode.

Discount
$\gamma = 0.999$ (the booking payoff lands at check-in, hundreds of minutes later).

What differs between the three is how they represent the state and pick the action. DQN feeds the state vector to a network that outputs a value $Q(s, a)$ for each of the four actions and takes the largest. PPO feeds the same vector to a network that outputs a *probability* for each action and samples one. MaskablePPO is PPO with a single change: before sampling, any action illegal in the current state has its probability forced to zero (equation (9)). Same state, same actions, same reward, same return, same discount; only the learner and the legality handling change.

A note on the baseline arms. The no-intervention baselines (the `PPO-base` and `DQN-base` bars) are a simpler MDP again: the state is an eight-number summary of a fixed planned walk, the action is just *stay* or *move on* (`Discrete(2)`), there is no booking, and the discount is 0.995

for the art lover and 0.997 for the tourist. They exist only to give the intervened agent a fair opponent: the same visitor, the same museum, with the masterpiece booking system switched off.

3.3 The whole thing, one line each

Algorithm	**State**	**Action**	**Reward**	γ
Q-learning	toy 5-tuple, one table	stay / move (legal)	toy per-step	0.99
Double-Q	toy 5-tuple, two tables	stay / move (legal)	toy per-step	0.99
DQN	25-dim booking vector	lead / book / pace, greedy	booking per-step	0.999
PPO	25-dim booking vector	lead / book / pace, sampled	booking per-step	0.999
MaskablePPO	25-dim booking vector	as PPO, illegal masked out	booking per-step	0.999

Reading across: the only things that ever change are which *problem* the algorithm is attached to (toy versus booking) and, within a problem, how it stores the state and screens the actions. That is the precise sense in which the five elements are properties of the problem, and the algorithm is the method you bring to it.

4 Learning from scratch: tabular Q-learning

Start with the smallest version of the museum: a toy map of twelve rooms (entrance, two corridors, the masterpieces, a couple of minor rooms, the exit). It is small enough that we can keep a giant lookup table with one entry for every (state, action) pair and simply fill in the numbers. This is *tabular* Q-learning, and it is the cleanest way to see the core idea before neural networks complicate the picture.

What the toy state actually is. Concretely, the toy state is a five-part label: which room you are in, which of ten time-slices of the day it is, how crowded Botticelli is (quantised to four levels), how crowded Leonardo is (four levels), and a small bitmask of which masterpieces you have already seen. Multiply those possibilities together and you get on the order of a hundred and twenty thousand distinct states, each with a handful of moves. That is small enough to keep every cell in memory and to visit each one many times over a training run, which is the entire reason the toy exists: it lets us watch the pure algorithm fill in a table and converge, with no neural network in the way to muddy what is happening.

4.1 The idea before the equation

Imagine a notebook with a row for every situation you could be in and a column for every move, and in each cell a running estimate of "how good is this move, here, in the long run?" At the start the notebook is blank (all zeros), so the agent has no idea what is good. It wanders, mostly at random, and every time it takes a step it learns one tiny lesson: "the move I just made led to an immediate reward, plus a next situation that my notebook says is worth such-and-such; so my old guess for that cell was a bit too high or too low, let me nudge it." Repeat that nudge a few hundred thousand times and the notebook fills in with remarkably accurate long-run values. That nudge is the entire algorithm.

4.2 The equation, piece by piece

The one-step update, applied every time the agent is in state s, takes action a, gets reward r, and lands in the next state s', is

$$Q(s,a) \;\leftarrow\; Q(s,a) \;+\; \alpha\Big[\; \underbrace{r + \gamma \max_{a'} Q(s',a')}_{\text{the TD target: a better estimate}} \;-\; \underbrace{Q(s,a)}_{\text{old estimate}} \;\Big]. \tag{3}$$

Take it apart. $Q(s,a)$ on the left is the cell we are updating. The arrow means "becomes." Inside the bracket is the *temporal-difference error*: the gap between a freshly computed, better estimate and the stale one we had. The better estimate, called the *TD target*, is $r + \gamma \max_{a'} Q(s',a')$: the reward we actually just received, plus the discounted value of the best thing we could do from the next state s'. The term $\max_{a'} Q(s',a')$ is the agent saying "from where I landed, I will assume I play optimally, so I take the largest value in that row." We subtract the old guess $Q(s,a)$ to get the error, and α (the *learning rate*, we use 0.1) controls how big a nudge we make: small enough to be stable, large enough to learn in reasonable time. This is called *bootstrapping*, because the estimate is updated toward a target that itself contains an estimate ($Q(s',\cdot)$). We pull ourselves up by our own bootstraps, and remarkably it converges.

Watching one cell learn. Suppose the cell $Q(s,a)$ currently holds 0 (we know nothing yet), the agent takes a, receives reward $r = 5$, and lands in a state s' whose best known move is worth $\max_{a'} Q(s',a') = 20$. With $\gamma = 0.99$ and $\alpha = 0.1$, the TD target is $5 + 0.99 \cdot 20 = 24.8$, the TD error is $24.8 - 0 = 24.8$, and the new value is $0 + 0.1 \cdot 24.8 = 2.48$. We did not jump all the way to 24.8; we moved a tenth of the way, because α keeps each lesson small so that one unlucky step cannot overwrite everything. The next time the agent visits the same cell it nudges again, and again, and the value creeps toward its true long-run worth. Notice what just happened conceptually: information about the good state s' (worth 20) flowed *backward* into the value of the earlier move a. Do this for every cell, millions of times, and value seeps backward from the exit and the masterpieces all the way to the entrance, until the table encodes a coherent long-run plan even though every single update only ever looked one step ahead.

Off-policy, and why that word matters. Q-learning is *off-policy*: the max in the target assumes the agent will act *optimally* from s' onward, even though, while exploring, it might actually take a random move next. In other words it learns the value of the best policy while behaving according to a more adventurous one. This is what lets it explore freely (try bad moves to learn they are bad) without corrupting the thing it is learning. The contrast with PPO later, which is *on-policy* and learns only about the policy it is currently running, is one of the deepest distinctions in the field, and it is the reason DQN can recycle old experience from a memory buffer while PPO cannot.

4.3 Two details that matter later

Exploration. If the agent always took what currently looks best, it would lock onto the first decent path it found and never discover better ones. So it uses *ε-greedy* action selection: with probability ε it acts randomly (explore), otherwise it takes the best-known move (exploit). We start at $\varepsilon = 1.0$ (all exploration) and decay it multiplicatively toward 0.01, so the agent is adventurous early and disciplined late. This randomness is seeded, and it is the *only* source of run-to-run difference on the deterministic toy, a fact we will need when we explain why two seeds give slightly different numbers.

Masking the max. A visitor can only step to an adjacent room. So in equation (3) the $\max_{a'}$ is taken only over *legal* moves from s', never over impossible ones. This is a small, free piece of correctness in the tabular world; it becomes a central design choice once we move to neural networks.

4.4 What it learns, and why

On the toy map, the crowds are *deterministic waves*: Botticelli peaks at a fixed time of day, Leonardo a bit later, and so on, the same every run. The agent is never told this schedule. Yet through equation (3) it discovers *temporal arbitrage*: visit a masterpiece in the valley before or after its crowd peak, not at the peak when a naive plan would send you. It learns this because the reward divides appreciation by a crowd term: the same painting pays far more when the room is empty. Q-learning, summing discounted rewards, sees that a small wait to enter off-peak buys a much larger reward, and the bootstrapping propagates that future gain back to the earlier "wait" action. That is the whole point of the toy: to show that timing, learned from nothing but trial and error, beats any fixed plan.

5 The optimist's curse: Double Q-learning

Q-learning has a famous, subtle flaw, and the project includes a second algorithm built specifically to fix it. Understanding the flaw is worth a page because it also explains a result that surprises people: on our toy, the "improved" algorithm does *not* win.

5.1 Why a single table is an optimist

Look again at the TD target $r + \gamma \max_{a'} Q(s', a')$. While learning, the values in the table are not yet correct; they are noisy estimates, some too high, some too low. Now ask what the max does to noise. If several actions have the same true value but your estimates are noisy, the max will, almost by definition, pick the one whose noise happened to be *positive*. So the maximum of noisy estimates is, on average, larger than the maximum of the true values. Formally this is *Jensen's inequality*:

$$\mathbb{E}\Big[\max_{a'} Q(s', a')\Big] \;\geq\; \max_{a'} \mathbb{E}[Q(s', a')]. \tag{4}$$

In words: the expected value of the maximum is at least the maximum of the expected values. The agent's TD target is therefore systematically too optimistic. This is *maximisation bias* or *overestimation*: the same noisy table is used both to *choose* the best next action and to *score* it, and that double duty inflates the estimate.

A coin-flip example you can feel. Imagine three moves whose true long-run value is exactly 0. Your estimates are noisy, so on a given step they might read $+3, -2, +1$. The truth is that the best move is worth 0, but $\max(+3, -2, +1) = +3$, so your target says "the best is worth $+3$." Another step the noise reads $-1, +4, 0$, and the max is $+4$. Average the maxima over many steps and you get a number comfortably above 0, even though no move is actually worth more than 0. You have convinced yourself the future is rosier than it is, purely because you always quote the luckiest of several noisy numbers. That is overestimation in one sentence, and it compounds: an inflated value for s' flows backward through bootstrapping and inflates earlier cells too. Double-Q breaks it because the table that *spotted* the lucky $+4$ is not the table asked *how much it is really worth*; the second, independently noisy table will, on average, quote something near the truth.

5.2 The fix: two tables and a coin

Double Q-learning (van Hasselt, 2010) breaks the self-flattery by keeping *two* tables, Q_A and Q_B, and on each step flipping a coin to decide which one to update. When updating Q_A, it uses Q_A only to *pick* the next action, but Q_B to *score* it:

$$Q_A(s,a) \;\leftarrow\; Q_A(s,a) + \alpha\Big[\, r + \gamma\, Q_B\big(s',\, \arg\max_{a'} Q_A(s',a')\big) - Q_A(s,a)\Big]. \tag{5}$$

The key is the inner part: $\arg\max_{a'} Q_A(s',a')$ asks Q_A "which action looks best?" and then $Q_B(s',\cdot)$ asks a *different, independently noisy* table "and what is that action actually worth?" Because the table that chooses is not the table that scores, the lucky positive noise that fooled a single table no longer lines up, and the overestimation largely cancels. At decision time the agent acts on the average $\frac{1}{2}(Q_A + Q_B)$. The symmetric update (swap A and B) runs when the coin lands the other way.

5.3 Why it does not help on our toy (and why that is the right result)

Here is the result that looks wrong at first: across five random seeds, vanilla Q-learning scores 83.3 ± 3.8 and Double-Q scores 83.1 ± 3.7. They are *statistically tied*; the variant neither helps nor hurts. The reason is that overestimation feeds on noise, and the toy has almost none. The crowd waves are deterministic and the rewards are a fixed function of (state, action), so once the table converges there is no random noise in the targets for the max to inflate. With no bias to remove, Double-Q's machinery is solving a problem that is not there. The small run-to-run wobble you see is not target noise; it is exploration noise, which way the ε-greedy coin sent the agent early in training, and that affects both algorithms equally.

This is exactly why we report both. The honest lesson is not "Double-Q is better" but "Double-Q earns its keep precisely when the environment is noisy." Hold that thought: when we get to DQN on the full, genuinely stochastic museum, the very same overestimation will return and will hurt, and the contrast will explain a real gap in the results.

6 The rules we compare against: the handcrafted baselines

To know whether learning is worth anything, we race the learned policy against five *handcrafted* policies on the toy: fixed rules a sensible person might write by hand. A learned policy that cannot beat "follow the guidebook route" has proven nothing.

- *default_path*: follow one fixed recommended route, the same for everyone.
- *random*: at each step pick a legal move uniformly at random.
- *greedy_least_crowded*: always step toward the emptiest neighbouring room.
- *greedy_value_ratio*: always step toward the room with the best art-per-crowd ratio, right now.
- *peak_avoidance*: a hand-built timing heuristic that tries to dodge the known peaks.

These are not learning algorithms; they are fixed strategies, so they need no training. They exist only as a yardstick. We will see in the results section that, strikingly, most of them score *negative*, worse than the random policy, and the explanation of why a "smart" rule loses to coin-flipping is one of the more instructive parts of the story.

7 When the table explodes: why we need neural networks

The lookup table worked because the toy had only twelve rooms and a handful of state features, about a hundred thousand cells in total, small enough to visit each many times and fill in. The real museum destroys that. It has ninety-eight rooms, roughly a hundred possible moves at each step, a continuous crowd density in every room, and an observation with several hundred numbers in it. A table over all of that would have more cells than we could ever visit, and almost every cell would stay blank forever. Worse, a table has no notion that two similar situations should have similar values; it treats every cell as unrelated, so nothing learned in one state helps in a nearby one.

What the full observation contains. For the deep agent the state is a vector of several hundred numbers, assembled fresh each minute: a one-hot code for the current room (a 1 in the slot for "where I am," zeros elsewhere), the crowd density in every room and whether that density is rising or falling, the time of day, an *egress slack* (how much time is left minus how many hops to the nearest exit), a visited-flag for every room, and, importantly, a per-room *appreciation-progress* value recording how much of each room's worth has already been banked. Several of these features earn their place only because earlier versions of the agent broke without them, the egress slack and the appreciation-progress especially, which is a story the failures section tells. The point for now is that the network reads this whole vector and, because it is a smooth function, can treat two similar vectors similarly, the generalisation a table can never manage.

The fix is *function approximation*. Instead of storing a separate number for every situation, we train a *neural network* to compute values or action-probabilities from the state. A network is just a flexible function with adjustable knobs (its weights): you feed in the state numbers, it multiplies and adds and bends them through a few layers, and out comes either a value or a probability for each action. Because it is a smooth function, it *generalises*: nudging the weights to be right about one crowded masterpiece room automatically makes it more right about other crowded rooms. We use small networks, two hidden layers of 128 or 256 units, which is plenty for this problem.

How a network learns, in plain terms. A neural network is a stack of *layers*. Each layer takes the numbers from the previous layer, multiplies them by its weights, adds them up, and passes the result through a simple bend (a nonlinearity) so the whole thing can represent curves and not just straight lines. The first layer reads the state; the last layer outputs what we want (a value, or a probability per action). Training means adjusting the weights so the outputs are good. We measure "how wrong" the network is with a *loss*, a single number that is large when the output is far from what it should be (for DQN the loss is the squared TD error; for PPO it is the clipped objective, flipped to a loss). Then we use *gradient descent*: compute, for every weight, the direction that would most reduce the loss (the *gradient*), and take a small step that way. Repeat on batch after batch of experience and the network slowly slides into weights that produce good outputs. So "the agent learns" really means "a loss is repeatedly nudged downhill by tiny weight adjustments." Nothing more mystical than that is happening, and keeping this picture in mind demystifies every training curve you will ever see: it is just a loss going down (and the corresponding reward going up) as the steps accumulate. Everything from here on replaces "look up the cell" with "ask the network," but the underlying ideas, value, return, temporal-difference error, are exactly the ones you already have.

8 Learning the policy directly: PPO

There are two broad ways to use a network. One is to keep estimating Q and act greedily, the neural descendant of Q-learning; that is DQN, and we cover it shortly. The other is to skip values as the main object and adjust the *policy* itself, the probabilities of each action, so that good actions become more likely. That second family is *policy-gradient*, and our workhorse, *PPO* (Proximal Policy Optimisation), lives there. PPO is the method behind the project's headline results, so it is worth understanding properly.

8.1 Policy gradient, the core idea

Forget values for a moment. Suppose the policy is a dial-box of probabilities: in each state it assigns some chance to each action, and we can turn the dials. The most direct imaginable learning rule is: run a whole visit, see how much total reward it earned, and then turn the dials so that the actions taken on a *good* visit become more likely and the actions taken on a *bad* visit become less likely. That is literally the original policy-gradient method, called *REINFORCE*, and its update has the shape

$$\theta \;\leftarrow\; \theta \;+\; \eta\, G\, \nabla_\theta \log \pi_\theta(a \mid s), \tag{6}$$

where G is the return the episode earned and $\nabla_\theta \log \pi_\theta(a \mid s)$ is the direction in weight-space that makes action a more probable. Multiplying by G means: if the episode was good (G large and positive), step in the direction that increases the probabilities of the actions you took; if it was bad, step the other way. Over many episodes the dials drift toward whatever earns reward. No values, no max, just "do more of what worked."

The trouble with the raw recipe is *variance*. Using the whole-episode return G to judge *every* action in that episode is terribly noisy: a single great booking can be buried inside an otherwise mediocre visit, and a clumsy move can ride the coat-tails of a lucky day. The learning signal is honest on average but jumps around so much that training is painfully slow. Two ideas tame it, and together they turn REINFORCE into a method that actually works at scale. First, instead of judging an action by the whole return, judge it by how much better things went than *expected*, which requires something that predicts the expectation. Second, take small, controlled steps so the noisy signal cannot blow the policy up on any single batch. The first idea gives us the critic and the advantage; the second gives us PPO's clipping. The next three subsections are exactly those two ideas, made precise.

8.2 Actor and critic

PPO trains two networks that cooperate. The *actor* is the policy $\pi_\theta(a \mid s)$: given a state, it outputs a probability for each action (θ denotes its weights). The *critic* is a value function $V_\phi(s)$: given a state, it estimates the return you expect from there. The actor proposes; the critic judges whether what happened was better or worse than expected, and that judgement is what tells the actor which way to adjust.

8.3 The advantage: was that better than expected?

The signal the actor learns from is the *advantage*, how much better an action turned out than the critic's baseline expectation. If you were in a state the critic valued at 100, took an action, and the outcome was worth 130, the advantage is +30: do that more. If it was worth 80, the advantage is −20: do that less. Estimating the advantage well is its own small art; PPO uses

generalised advantage estimation (GAE), which blends short- and long-horizon estimates:

$$\hat{A}_t \;=\; \sum_{l=0}^{\infty} (\gamma\lambda)^l \, \delta_{t+l}, \qquad \delta_t \;=\; r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t). \tag{7}$$

Here δ_t is a one-step surprise: the reward plus the discounted value of where you landed, minus the value of where you were; a positive δ_t means that single step beat expectations. GAE sums these surprises over the future, each discounted by $\gamma\lambda$. The extra knob λ (we use 0.98) trades off noise against bias: near 1 it looks far ahead (less biased, a bit noisier), near 0 it trusts the critic's one-step estimate (smoother, more biased). A high λ here gives the actor a clean, low-bias sense of which booking decisions truly paid off many minutes later.

The advantage, with numbers. Suppose the critic valued the current state at $V(s_t) = 100$, the agent took an action, received $r_t = 5$, and the critic values the state it landed in at $V(s_{t+1}) = 120$. The one-step surprise is $\delta_t = 5 + 0.999 \cdot 120 - 100 = 24.88$: things went markedly better than the critic expected, so the advantage is strongly positive and PPO will make that action more likely. Had the next state been valued at only 80, we would get $\delta_t = 5 + 0.999 \cdot 80 - 100 = -15.08$, a negative advantage, and PPO would make the action *less* likely. GAE then chains these one-step surprises over the rest of the episode (each discounted by $\gamma\lambda$), so an action is credited not only for the immediate surprise but for the run of good or bad surprises it set in motion. That is exactly the right notion of credit for a booking whose benefit only unfolds across the whole visit: the booking action inherits the advantage of every well-timed check-in it later made possible.

8.4 The clipped objective, and why "proximal"

Naively, you would push the policy hard toward every action with positive advantage. The danger is overshooting: one big greedy update can wreck a policy that took ages to shape, and on-policy methods cannot easily undo it. PPO's whole contribution is a brake. Define the *probability ratio* $\rho_t = \pi_\theta(a_t \mid s_t) \,/\, \pi_{\theta_{\text{old}}}(a_t \mid s_t)$, how much more (or less) likely the new policy makes the action than the policy that collected the data. PPO maximises

$$L^{\text{CLIP}}(\theta) \;=\; \mathbb{E}_t\Big[\min\left(\rho_t \, \hat{A}_t,\; \text{clip}(\rho_t,\, 1-\epsilon,\, 1+\epsilon)\, \hat{A}_t\right)\Big]. \tag{8}$$

Read it as a careful instruction. We want to increase probability for positive-advantage actions, that is the $\rho_t \hat{A}_t$ term. But $\text{clip}(\rho_t, 1-\epsilon, 1+\epsilon)$ refuses to let the ratio move outside $[1-\epsilon, 1+\epsilon]$ (with $\epsilon = 0.2$, outside $[0.8, 1.2]$), and the min takes the less optimistic of the clipped and unclipped versions. The effect: once an action's probability has moved about twenty percent, the objective stops rewarding further movement on that batch, so no single update can lurch. "Proximal" means each new policy stays close to the last one. This self-imposed caution is the reason PPO is *stable*, and stability, as the results will show, is most of why it wins.

The clip, with numbers. Say an action had a positive advantage $\hat{A}_t = 10$, and the update has already raised its probability so that $\rho_t = 1.5$ (it is now 1.5 times as likely as under the old policy). The unclipped term would reward us $1.5 \cdot 10 = 15$ for pushing even harder. But the clip caps ρ_t at 1.2, giving $1.2 \cdot 10 = 12$, and the min picks the smaller, 12. Because the clipped value no longer grows as ρ_t rises past 1.2, the gradient there is zero: PPO simply stops paying us to inflate this action further on this batch. The action will keep rising, but only a little per batch, across many batches, never in one reckless leap. For a negative advantage the clip works symmetrically on the downside. This is the precise mechanism behind "small, controlled steps," and it is why a single noisy batch cannot wreck a policy that took a million steps to shape.

8.5 The training loop, in practice

PPO does not learn from one step at a time. It runs the current policy for a fixed stretch, a *rollout* of n_{steps} environment steps (we use 2048), recording every (s, a, r) and the critic's value. It then computes the advantages (equation (7)) for that whole batch and improves the policy on it for several passes, n_{epochs} (we use 8 to 10), chopping the rollout into mini-batches of a few hundred. Then it throws the data away and collects a fresh rollout with the improved policy. This rollout-then-refine rhythm is what "on-policy" looks like in practice: the data always reflects the current policy, which is why it cannot be hoarded the way DQN hoards its replay buffer. One more ingredient keeps exploration alive: an *entropy bonus* (a small reward, weight ≈ 0.01, for keeping the action probabilities spread out rather than collapsing to a single certain choice too early). Without it a policy can prematurely commit to a mediocre habit; with it, the agent keeps a little curiosity until the evidence is in. The far-sighted discount ($\gamma = 0.999$ for booking), the low-bias advantage ($\lambda = 0.98$), the clip ($\epsilon = 0.2$), and this entropy nudge are the handful of dials that define our PPO; none of them is exotic, and the defaults work because masking has already made the problem clean.

8.6 How PPO differs from Q-learning

Two differences are worth stating plainly. First, PPO is *on-policy*: it learns from data collected by the current policy, then throws it away, whereas Q-learning and DQN are *off-policy* and reuse old experience. On-policy data is more expensive but better matched to what the policy is currently doing. Second, PPO adjusts probabilities smoothly and never takes a max over noisy value estimates, so the overestimation curse of section 5 simply does not arise for it. That single fact will explain a lot about why PPO is steady where DQN is jumpy.

9 Telling the agent the rules of the building: action masking

The museum is a graph: from any room only a few moves are physically possible, but the action head of the network has about a hundred outputs, one per possible room. Without help, the policy spends most of its probability on moves that do not exist and has to *learn*, from penalties, that they are illegal, wasting effort on the obvious.

Action masking hands the agent the rulebook for free. Before turning the network's raw scores (logits) into probabilities, we set the score of every illegal action to minus infinity, so it receives exactly zero probability:

$$\pi_\theta(a \mid s) \;=\; \frac{\exp(z_a)\,\mathbb{1}[a \in \mathcal{A}(s)]}{\sum_{a' \in \mathcal{A}(s)} \exp(z_{a'})}, \tag{9}$$

where z_a is the network's score for action a, $\mathcal{A}(s)$ is the set of legal actions in state s, and $\mathbb{1}[\cdot]$ is 1 for legal actions and 0 otherwise. The denominator sums only over legal actions, so the probabilities of the legal moves renormalise to one and every illegal move is impossible by construction. *MaskablePPO* is just PPO with this mask applied identically during training and evaluation.

This sounds like a minor convenience. It is in fact the single most important architectural choice in the project, because it removes an entire learning problem ("which moves are even allowed?") and lets every scrap of gradient go into the decision that actually matters.

Why the unmasked version is genuinely harder, not just messier. Without masking, the network must *learn* to avoid illegal moves from a penalty, and that has three costs that compound. It wastes capacity: probability mass and gradient go into suppressing the eighty-odd

impossible actions instead of ranking the handful of real ones. It slows learning: early on, most sampled actions are illegal, so most steps teach the dull lesson "that was not allowed" rather than anything about good play. And it adds noise to the advantage estimates, because an episode's return now mixes the signal of good decisions with the static of accidental illegal moves and their penalties. Masking is, mathematically, exactly equivalent to handing the agent a different, smaller, state-dependent action space, the legal one, so none of these costs is ever paid. That is why we describe it not as a tweak but as changing the problem the network has to solve. The results section quantifies exactly how much it buys, and, revealingly, *where*: only where the underlying decision is hardest.

10 The value-based alternative: DQN

For contrast, the project also trains *DQN* (Deep Q-Network), the neural version of Q-learning. It keeps the value-based spirit: learn $Q(s, a)$ with a network Q_θ, then act greedily. It minimises the squared temporal-difference error, the same target as equation (3) but fit by gradient descent:

$$L(\theta) \;=\; \mathbb{E}\Big[\big(\, r + \gamma \max_{a'} Q_{\theta^-}(s', a') - Q_\theta(s, a) \,\big)^2\Big]. \tag{10}$$

Two tricks make this stable enough to work with a network. A *replay buffer* stores past experience and replays random batches of it, so the network is not whiplashed by the latest correlated steps. And a *target network* Q_{θ^-}, a slowly updated copy of the weights, supplies the value of s' in the target, so the thing we are chasing does not move every single step. We give DQN a buffer of a hundred thousand to two hundred thousand transitions, a slow target update, and a sensible exploration schedule.

The deadly triad. There is a well-known warning in reinforcement learning that three ingredients, when combined, make learning unstable: *bootstrapping* (updating an estimate toward a target that contains another estimate), *function approximation* (a neural network rather than an exact table), and *off-policy* learning (training on data from a different, older policy via the replay buffer). This trio is called the *deadly triad*, and DQN has all three at once. None of them is individually fatal, the tabular toy had bootstrapping and off-policy learning and was fine, but together, on a hard noisy problem, they can let errors feed on themselves: a wrong value gets bootstrapped into neighbouring states, the network generalises the error to states it has not even seen, and the stale buffer keeps reinforcing it. PPO sidesteps the triad entirely: it is on-policy and does not bootstrap a max, which is a large part of why it is the calmer learner.

The overestimation, returning. Notice the $\max_{a'}$ sitting right there in equation (10). This is the same maximisation that caused overestimation in section 5, and plain DQN does *not* use the double-table fix (its deep cousin *Double-DQN* exists precisely to add one, by choosing the next action with the online network but scoring it with the target network, exactly the Double-Q trick of section 5.2). On a deterministic toy this did not matter, which is why Double-Q tied vanilla there. On the full museum, where the crowd is genuinely stochastic and the value targets are therefore noisy, it matters a great deal. Our DQN is also unmasked and chases a moving target with a big function approximator. Keep all of this in mind: bootstrapped overestimation, the deadly triad, no masking. It is precisely the recipe for the crowd-fragile, high-variance behaviour we will see in the numbers, and it is the live, full-scale echo of the toy lesson that Double-Q only earns its keep under noise.

11 Exploration versus exploitation, the running tension

Every method in this guide faces the same dilemma at every step. Should the agent take the move that looks best on current evidence (*exploit*), or try a move it is unsure about to find out whether it is secretly better (*explore*)? Pure exploitation locks onto the first decent habit and never improves, because it never gathers evidence about the roads not taken. Pure exploration learns a lot but never cashes any of it in. Good learning explores heavily early, when it knows little, and shifts toward exploiting as the evidence piles up.

The tabular and value-based methods (Q-learning, DQN) manage this with *ε-greedy*: with probability ε act at random, otherwise take the best-known move. We start ε near 1 (almost all exploration) and decay it toward near 0 (almost all exploitation), so the agent is adventurous as a beginner and disciplined as it matures. PPO reaches the same goal by a different door, the *entropy bonus*: a small reward for keeping the action probabilities spread out rather than collapsing onto one near-certain choice. Entropy is just a measure of how undecided a distribution is, high when the options are balanced, low when one dominates, so paying for entropy is literally paying the policy to stay a little open-minded until the evidence justifies committing.

These are two costumes for one idea, and getting it wrong is behind real failures. The do-nothing trap (the free decline) is exactly an exploration failure: the safe action paid off immediately, the agent stopped trying the booking alternative before it ever experienced the delayed reward, and so it never learned that booking was better. Whenever an agent "gets stuck" on a mediocre habit, exploration is the first thing to interrogate.

12 The three deep methods at a glance

Before reading the results, it helps to line up the three networked methods on the axes that actually drive their behaviour. Everything in the table traces back to the sections above, and every row will explain a feature of the numbers.

	DQN	**PPO**	**MaskablePPO**
Family	value-based	policy-gradient	policy-gradient
Learns	$Q(s, a)$, acts greedily	the policy directly	the policy directly
On- or off-policy	off-policy (replay)	on-policy	on-policy
Takes a max over noisy values?	yes (the liability)	no	no
Knows legal moves for free?	no	no	yes (masking)
Main strength here	sample-reuse	stable updates	stable + clean action set
Main weakness here	overestimation under noise	must learn legality	none of note

Read the table as a set of predictions. The "yes" in the max-over-noisy-values row is why DQN alone suffers overestimation on the noisy crowd. The "no" in the legal-moves row for plain PPO is why it wastes effort and stumbles exactly where the decision is hardest. And the clean column for MaskablePPO, stable updates and a free-of-charge legal action set, is why it wins. The results section is essentially this table, made quantitative.

13 The scaffolding: curriculum, common random numbers, and seeds

Three pieces of machinery are not algorithms but make the comparisons trustworthy, and the project's conclusions lean on them.

Curriculum. Training an agent directly against the full five-thousand-person crowd is so noisy early on that it learns slowly. So for the navigation agent we ramp the difficulty: a first stage at about a third of the peak crowd, then two-thirds, then the full crowd, always over a complete museum day. Learning the easy version first and then transferring is faster and more reliable, the same reason you would not teach someone to drive in rush-hour traffic on day one. The deeper justification is about signal-to-noise. A beginner policy makes mostly poor choices, and in a packed museum those poor choices are buried under heavy crowd randomness, so the learning signal is weak and the agent flails. In a light crowd the same poor choices produce clearer, less noisy consequences, so the basics (where the masterpieces are, how to pace, when to leave) lock in quickly; once they are in place, the agent can withstand the full crowd's noise and refine. The ordering matters because skills learned early are a scaffold for the harder ones, not a separate lesson to be unlearned. Note that the booking agents, whose decision is simpler than full navigation, do not need the ramp and are trained directly at each crowd level.

Common random numbers. To compare two policies fairly, they must face the *same* crowds. We fix six crowd scenarios (seeds 900000 to 900005) and score every policy on those identical six. This is *common random numbers*: because both policies meet the same randomness, any difference in their scores reflects the *policy*, not the luck of which crowd each happened to draw. It sharply reduces the noise in a comparison.

Why it helps, concretely. Imagine policy A scores 3700 on one day and policy B scores 3600 on another day. Is A genuinely better, or did it just draw an easier crowd? You cannot tell; the difference could be all luck. Now force both to play the *same* six crowd scenarios and compare them scenario by scenario: if A beats B on the very same crowd, the gap is about the policies, because the only thing that changed is the policy. It is the logic of a fair race, everyone runs the same track on the same day, so the finishing order means something. This is why our evaluation crowds (seeds 900000 through 900005) are frozen and identical for every model in the grid, and it is what lets us read a small gap between two algorithms as real rather than noise.

Multiple seeds. A single training run is one sample of a random process (random network initialisation, random exploration, random mini-batches). One run can be lucky or unlucky, so a single number can mislead. We therefore train each cell of the comparison under five different *training seeds* $(1, 3, 7, 42, 202606)$ and report the mean and the standard deviation across them. The mean is our best estimate; the standard deviation tells us how reliable that estimate is. As we will see, this matters: one cell of the grid looks like a clean win on a single seed but is revealed, across five, to be a coin-flip between a large win and a collapse.

How to read a training curve. Several figures in the project plot a learning curve, and they all read the same way. The horizontal axis is training progress (episodes for the toy, environment steps for the deep agents); the vertical axis is the return the policy achieves. A healthy curve starts low (the untrained agent is bad), climbs as the loss is nudged downhill batch after batch, and then *plateaus* once the policy stops improving, that flat top is *convergence*, and it is the part we care about, not the noisy beginning. The shaded band around the line is variability (a rolling

confidence interval, or the spread across seeds): a narrow band means a reliable result, a wide one means a fragile one. Two warnings the curves teach. First, read the plateau, not the single highest point; the peak of a noisy curve is a lucky fluctuation, not the policy's true level. Second, a curve that climbs nicely in *training* reward can still hide a bad *evaluation* policy if training and evaluation differ (the very issue behind one of our failures, where a stochastic policy trained well but its greedy version evaluated poorly). A learning curve is a thermometer, not a verdict; the verdict comes from the common-random- number evaluation above.

14 The decision that actually matters: the booking problem

Early in the project the agent was asked to learn how to *walk* the museum. That was a mistake we corrected, and the correction is itself a lesson worth stating: a real visitor has a map and is guided by staff, so optimising the walk is a shortest-path puzzle dressed up as learning, with a trivial, fully observed state. The genuinely uncertain, consequential decision is the *booking*, and reframing the task around it is what made the project's centerpiece. The walking then happens underneath as automatic pacing.

Choosing the MDP is the modelling. It is tempting to think the hard part of a reinforcement-learning project is the algorithm. It is not. The hard part is deciding *what decision the agent is even making*, that is, choosing the state, the actions, and the reward. We spent months making a navigation agent work, and the brittle result kept whispering that something was wrong. The realisation was that we had chosen the wrong MDP: walking a known map with a guide is barely a decision at all, so no algorithm, however good, could make it interesting. Moving the action from "which room next" to "when, and how far ahead, to book" turned a trivial shortest path into a genuine sequential decision under uncertainty with a delayed payoff, the kind of problem where learning earns its keep. The same equations and the same PPO then produced a result worth reporting, not because the algorithm got better but because the question did. If you take one methodological lesson from this guide, let it be that the formulation is more than half the work: a beautiful optimiser pointed at the wrong decision optimises nothing.

14.1 A two-phase episode

Each episode now has two phases. In *Phase 1 (booking)*, before the visit, the agent chooses a *lead time*, how many days in advance to book, from $\{1, 7, 21, 35\}$, and then, for each of the three gated masterpieces, whether to take the offered slot. In *Phase 2 (the visit)*, it walks the museum and must *check in* at each painting inside the time window it booked. The discount is set high, $\gamma = 0.999$, because the reward for a good booking does not arrive until check-in, which can be hundreds of minutes into the visit; a far-sighted agent is mandatory.

One full episode, narrated. Picture a packed day. In Phase 1 the agent, reading that the museum is full, chooses a lead time of 35 days, because the fill curve tells it that anything shorter will not secure the popular slots. It accepts the offered windows for Botticelli, Leonardo, and Raphael, spread across the day. So far it has earned almost nothing; the payoff is still hundreds of steps away. Phase 2 then begins at the entrance. The agent walks room by room (every move a legal neighbour), pacing itself so that it arrives at Botticelli inside its ten-minute window, where it finally banks the appreciation and a check-in bonus; it continues to Leonardo and Raphael in their windows, drops to the first floor for Caravaggio, and leaves before closing. Only now, at each check-in, does the large reward land, and the high discount carries that reward all the way back to justify the lead-35 choice made before the visit even started. That backward flow of

credit, from a check-in late in the afternoon to a booking made weeks in advance, *is* the learning problem in miniature, and it is why the next subsection is about credit assignment.

14.2 The reward, component by component

The per-minute reward is a sum of terms, and reading them is the fastest way to understand what the agent is being asked to want:

$$r \;=\; \underbrace{r_{\text{art}}}_{\text{appreciation}} \;+\; \underbrace{r_{\text{closing}}}_{\text{get-out pressure}} \;+\; \underbrace{r_{\text{completion}}}_{\text{finished a must-see}} \;+\; \underbrace{r_{\text{checkin}} + r_{\text{wait}}}_{\text{booking discipline}} \;+\; \underbrace{r_{\text{time}}}_{\text{small step cost}} \;. \tag{11}$$

The central term is the appreciation,

$$r_{\text{art}} \;=\; \text{importance} \times \underbrace{\frac{1}{1+\alpha\,\text{density}^2}}_{\text{crowd discount}} \times \underbrace{\text{novelty}}_{\text{satiation}}, \tag{12}$$

which says a room pays its *importance*, scaled down when it is crowded (the crowd discount, with α small so a crowded masterpiece still beats skipping it, $\alpha = 0.5$ for the art lover, 1.0 for the tourist), and scaled down the longer you have already stood there (*novelty* decays, so value *satiates* and the agent eventually moves on).

The crowd discount, with numbers. Take a masterpiece of importance 10 and the art lover's crowd-aversion $\alpha = 0.5$. In an empty room (density 0) the appreciation is $10 \times \frac{1}{1+0} = 10$ times novelty. In a room jammed to twice capacity (density 2) it is $10 \times \frac{1}{1+0.5\cdot 2^2} = 10 \times \frac{1}{3} \approx 3.3$ times novelty: the same painting pays a third as much when packed. The number that matters is that it is *still positive*, so a crowded Botticelli still beats skipping it. This is a deliberate calibration. An earlier, harsher crowd term drove the discounted value below the reward of simply walking past, and then the optimiser rationally skipped the masterpiece, producing a "fake art lover" who speed-ran the famous rooms. Shrinking α until a crowded masterpiece is still worth entering is what made the modelled connoisseur behave like one. As for novelty: it decays with each minute you stay, so after enough time in a room the marginal minute pays almost nothing, and moving on becomes the better action without any explicit "you are bored now" rule. Satiation, not a timer, is what ends a visit to a room. The other terms each fix a specific failure: r_{closing} is a pressure that grows as closing nears and you are far from an exit, so the agent leaves on time; $r_{\text{completion}}$ is a one-off bonus for fully appreciating a must-see; r_{checkin} rewards arriving inside a booked window and r_{wait} gently penalises idling for one; r_{time} is a small per-step cost so dithering is never free. The booking phase adds a few more: a penalty for declining a masterpiece (since these profiles always want all three), a no-show penalty for booking a slot and missing it, and, decisively, an *immediate off-pace penalty* $\text{mismatch}_k \cdot |\text{window} - \text{estimate}|$ paid at booking time. That last one matters for a reason we come to below.

14.3 The fill curve, and why "book early" is a real decision

Booking is only interesting because slots are scarce on busy days. A slot is available only if the lead time is long enough:

$$\text{lead} \;\geq\; \text{lead}_{\max} \times \text{popularity} \times \frac{\text{daily visitors}}{2500}. \tag{13}$$

The busier the day, the larger the right-hand side, so the longer ahead you must book to secure a popular masterpiece. Book late on a packed day and the good slots are simply gone, and you miss the painting entirely, which is a large loss of reward. Book early and you lock in well-spaced windows. There is no downside to booking early; the only question is how early, and the agent has to *discover* that the answer rises with the crowd. This is the heart of the centerpiece result.

14.4 Why this is hard: the credit-assignment problem

Booking is a textbook case of the hardest thing in reinforcement learning, *credit assignment* across a long delay. The agent picks a lead time in the first few steps of the episode. The consequence, did it actually get into the masterpiece inside a good window, or did it arrive to find the slot mistimed and waste the visit, does not reveal itself until hundreds of minutes later at check-in. So the reward that should teach "you booked too late" is separated from the booking action by a vast stretch of unrelated walking. A far-sighted discount ($\gamma = 0.999$) is necessary to carry the signal back that far, but it is not sufficient: a faint signal smeared across four hundred steps produces a tiny, noisy gradient on the booking action, and early versions of the agent only half-learned the lesson (a blurry mix of lead times, two-and-a-bit masterpieces secured at the packed crowd). The cure was to manufacture a *local proxy* for the distant outcome: the immediate off-pace penalty $\text{mismatch}_k \cdot |\text{window} - \text{estimate}|$, paid right at booking time, which is large exactly when the booked window is far from when the agent can realistically arrive. Now the agent feels a consequence *immediately*, correlated with the distant one, and the gradient has something nearby and strong to follow. This single trick is what turned "book early" from half-learned to crisply learned, and it is a beautiful illustration of a general principle: when the true reward is too far away to teach, engineer an honest local signal that points the same way.

14.5 Two modelling choices worth being explicit about

Two design decisions shape these results and deserve to be named, because a careful reader should know which parts are learned and which are assumed.

First, the agent is *not* allowed to decline all three masterpieces. We tried leaving a free "decline" button and both visitor types collapsed to declining everything (we explain why in the failures section: a free skip is a classic do-nothing trap). We treat "always books the three masterpieces" as a *type characterisation*, a fact about who these visitors are, nobody flies to Florence and skips all of Botticelli, Leonardo, and Raphael, rather than something the agent must rediscover. The genuinely learned decision is the *lead time*, and that is what we report.

Second, the agent needs an estimate of when it will arrive at each masterpiece in order to judge a booking, and that estimate is computed once from reference walks and then *frozen* during training. The reason is subtle and important: if the arrival estimate were updated online from the agent's own past arrivals, it would become a function of the policy that is itself feeding the policy, a self-referential loop that drifts (again, see the failures section). Freezing it breaks the loop. This is an assumption, but a transparent one, and the qualitative lesson (book earlier when busier) does not depend on its exact value.

15 Reading the results through the theory

This is the section the rest of the guide was building toward. For each finding, we say what the number is and then why the theory above forces it to be that way. If you understand these, you can look at any of the project's tables and explain it.

15.1 Why Q-learning and Double-Q tie on the toy (83.3 vs 83.1)

Because overestimation, the only thing Double-Q fixes, is born of noise, and the toy has none. The crowd waves are deterministic and the rewards are a fixed function of (state, action), so at convergence there is no random jitter in the TD target for the max in equation (3) to inflate (equation (4) bites only when the estimates are noisy). Double-Q's two-table machinery therefore

cancels a bias that is not present, changing nothing on average. The tiny gap between runs is exploration noise, which both share equally. Predicted consequence: the moment we move to a genuinely stochastic environment, this tie should break in a way that hurts the single-table method, and it does, in DQN below.

15.2 Why the handcrafted baselines lose, and the "smart" ones lose worst

The learned policies score around 83; the best hand rule, *random*, manages only about 35, and the structured rules are *negative*: *peak_avoidance* ≈ -34, *greedy_least_crowded* ≈ -43, *default_path* ≈ -61, *greedy_value_ratio* ≈ -72. The negatives come from the closing-time and step-cost terms in the reward: a fixed rule that mistimes its visit gets caught wandering as closing nears and pays the egress pressure, while collecting little appreciation because it stood in crowds. The reason the *structured* rules do worse than *random* is the sharpest point: *greedy_value_ratio* marches straight at the highest-value room *regardless of when*, so it reliably arrives at Botticelli at the peak, exactly the worst moment, and the crowd discount in equation (12) guts its reward. A rule that is confidently wrong about timing is worse than one that stumbles around, because the random walker at least samples some off-peak minutes. This is the toy's entire moral: *when* you visit dominates *whether*, and only the learner discovers it.

15.3 Why MaskablePPO wins everywhere

MaskablePPO is best or tied-best in every cell of the grid. Two of its properties from the sections above combine. First, masking (equation (9)) deletes the entire sub-problem of learning which moves are legal, so all of its capacity goes into the booking and pacing decision. Second, the clipped objective (equation (8)) makes its updates small and non-destructive, so it converges to a good policy and stays there rather than thrashing. It has no max over noisy values, so it never suffers the overestimation that destabilises DQN. Steady optimisation of the right, constrained problem is simply hard to beat.

15.4 Why unmasked PPO matches at easy crowds but collapses at the packed tourist

This is the cleanest demonstration of what masking buys. Plain PPO equals MaskablePPO at the quiet and busy tourist ($\approx$ 3686 and 3695), then at the packed tourist it collapses to 2093, *below* the no-booking baseline of 2680, while MaskablePPO holds 3685. At low crowd the booking problem is forgiving: many lead times work, so masked and unmasked policies converge to the same easy optimum. At the packed crowd the fill curve (equation (13)) is at its tightest, only a long lead and precise pacing secure the slots, and that knife-edge is exactly where wasting probability on illegal or pointless moves is fatal. The unmasked policy cannot reliably hold the tight requirement and degrades into something worse than not booking at all. The masking is decisive precisely where the decision is hard, which is why the gap appears only in that corner of the grid and is reproducible across all five seeds.

15.5 Why DQN is crowd-fragile and high-variance

DQN's value-based baseline on the art lover falls from about 4527 at the quiet crowd to 2044 at the packed crowd, and its bars carry the widest error bars in the figure, with standard deviations up to roughly ± 2600 across seeds. This is section 5's overestimation returning to collect its debt. The full museum is genuinely stochastic, so the value targets are noisy; the max in equation (10) then inflates them (Jensen again), and plain DQN, unlike Double-Q, has no mechanism to cancel

the bias. The inflation grows with the crowd because a busier museum is a noisier one, which is why the collapse tracks the crowd. And because value-based bootstrapping off a moving target is sensitive to initialisation and replay order, different seeds land in very different places, hence the large spread. Everything that made DQN look fine on the deterministic toy is exactly what makes it fragile here; the contrast is the point of including it.

15.6 Why "book early when busier" emerges from learning

The learned lead time rises with the crowd: the art lover goes from booking 7 days ahead when quiet to 35 days ahead when busy; the tourist from 7 to about 21. The fill curve makes late booking on a packed day forfeit the masterpieces, a large reward loss under equation (12), and the far-sighted discount $\gamma = 0.999$ lets that distant loss reach back to the booking action. But a delayed loss alone is a weak teacher, the gradient for a payoff hundreds of steps away is faint, which is why early versions only half-learned it. The immediate off-pace penalty $\text{mismatch}_k \cdot |\text{window} - \text{estimate}|$ is what fixed it: it puts a small, local cost at booking time that correlates with the distant outcome, giving the gradient something nearby to follow. The behaviour is not hand-coded; it is the optimum of the reward, made findable by one well-placed local signal.

15.7 Why the tourist is rock-stable but the art lover at max crowd is a coin-flip

This is the subtlest reading, and the one the five seeds exist to reveal. The tourist's intervened scores have essentially zero spread across seeds at every crowd, and its lift is a steady +36% to +38%. The art lover's lift is healthy at the quiet and busy crowds (+39%, +29%) but at the packed crowd it averages only +20% *with a huge standard deviation* of about ±1900: four of the five seeds reach roughly +41%, and one seed collapses to a score well below the baseline.

The difference is the shape of the optimisation landscape for each profile. The tourist has short dwell times, cares about only a handful of recognisable rooms, and is only mildly crowd-averse, so its optimal visit is simple and essentially unique; every seed rolls down to the same solution, and with deterministic evaluation that yields identical scores. The art lover is the opposite: long dwells, value in many rooms, and strong crowd-aversion, so at the packed crowd the optimum is a knife-edge, book the longest lead *and* pace a long, crowd-sensitive visit so every window is met before closing. That narrow optimum sits next to a basin of failure, and which one a given training run falls into depends on its seed. Four seeds find the knife-edge (+41%); one slips into the failure basin and collapses. Averaging gives the modest +20% with the wide band. The honest way to report this cell, and the way the figure now shows it, is the mean with its error bar: the intervention usually delivers a large gain for the art lover even at maximum crowd, but at that single hardest cell the result is not guaranteed, and the spread says so.

15.8 Why all three masterpieces are secured at every crowd

Across every cell the agent ends up with 3/3 masterpieces booked and checked in. This is the do-nothing trap, resolved by design (section 14.5): because declining is not an available action, the only question the agent optimises is the lead time, and with the fill curve the way to keep all three is simply to book early enough. So securing all three is not a surprising feat of intelligence; it is the consequence of removing an unrealistic escape hatch and letting the agent solve the one real problem, when, rather than a fake problem, whether.

15.9 Why the learned routes are clean (zero teleports)

When we audit the walks, every room-to-room move is a physical one-hop neighbour; there are no jumps across the map. This is action masking doing its quiet job: the mask makes a non-adjacent move literally impossible (zero probability, equation (9)), so a teleport cannot occur even by accident. A reader who did not know about masking might be impressed that the agent "learned the floor plan"; in truth it was handed the floor plan, which is exactly the point of masking and exactly why we do not have to spend the agent's capacity teaching it geography.

15.10 Why lead times jump in discrete steps

The learned lead time does not glide smoothly; it sits at 7, then jumps to 35, with little in between. That is because the action set offers only $\{1, 7, 21, 35\}$ days, and the fill curve (equation (13)) is a threshold: below the required lead the slot is simply unavailable, above it the slot is secured. So the agent has no reason to pick a middle value, it picks the smallest offered lead that clears the threshold for the current crowd. The discreteness of the curve in the figures is therefore a faithful picture of a discrete decision, not a plotting artifact, and the jump to 35 at the packed crowd is the agent reporting that nothing shorter would clear the threshold.

16 One-page cheat-sheet

Method	**What it is, in one line**	**Why its number looks the way it does**
Tabular Q-learning	Fill a table of long-run move values by trial-and-error nudges.	Beats every hand rule (≈ 83) because it learns to visit off-peak; timing dominates value.
Double Q-learning	Q-learning with two tables to cancel optimism.	Ties vanilla (≈ 83) on the toy: deterministic, so no overestimation to remove.
Handcrafted rules	Fixed heuristics, no learning.	Mostly negative; the confidently mistimed ones (greedy value) lose worst, below random.
PPO	Adjust the policy directly with small, clipped steps.	Stable; no noisy max, so it never overestimates. The backbone of the wins.
MaskablePPO	PPO told which moves are legal before choosing.	Best or tied everywhere; masking frees all effort for the real decision.
DQN	Neural Q-learning with replay and a target network.	Crowd-fragile, high-variance: the overestimation that was harmless on the toy bites under a noisy crowd.
Action masking	Zero out illegal actions before the softmax.	Decisive only at the packed tourist, where the booking constraint is tightest.
Curriculum / CRN / seeds	Ramp difficulty; fix the test crowds; repeat runs.	Make the comparison fair and reveal the art-lover/max-crowd cell as a coin-flip, not a clean win.

The single thread through all of it: value-based methods take a maximum over their own noisy estimates and so flatter themselves when the world is noisy; policy-gradient methods nudge probabilities and do not. That one difference, plus the free gift of action masking, explains the ranking of the algorithms, and the shape of each profile's reward landscape explains the spread of the numbers within the winner.

17 Failures as reinforcement-learning lessons

Most of the project was spent being wrong in instructive ways, and each mistake maps onto a named, general phenomenon in reinforcement learning. Understanding these is as valuable as understanding the algorithms, because they are the traps you fall into whenever you try to make an agent want the right thing. We present five, each as a symptom, the underlying concept, and the fix.

17.1 The unlearnable reward (sparse, all-or-nothing signals)

Symptom. The agent refused to leave the museum; it would squat in a room until the day ended. The obvious fix, "zero out all your reward if you fail to exit," made it *worse*.

Concept. A reward that is all-or-nothing at the very end is *sparse* and, worse, nearly flat: almost every policy fails to exit, so almost every policy gets the same zero, and a function that is flat across policies has *no gradient*. Gradient descent needs a slope to walk down; an all-or-nothing terminal wipe gives it a cliff surrounded by a plateau, and from the plateau nothing points toward the cliff edge. The agent cannot learn what it cannot feel a gradient toward.

Fix. Replace the wipe with a *dense* signal: a per-step egress pressure that grows smoothly as closing nears and as you get further from an exit, plus a finite (not infinite) penalty for not leaving and a bonus for leaving. Now every step toward the door is rewarded a little, so there is a continuous slope leading out. The lesson, which recurs throughout the field, is that a reward must be *shaped* so that better behaviour is always at least slightly better-scoring; sparse rewards are correct but often unlearnable.

17.2 Acting on what you cannot see (partial observability)

Symptom. The rule "stay in a room until you have appreciated it enough, then move on" never stabilised; the agent over- and under-stayed seemingly at random, even with a sensible reward.

Concept. The right action depended on *how much value the agent had already extracted from the current room*, but that quantity was not in the observation. The decision was therefore not *Markov* in the state the policy could see (recall section 2): the agent was being asked to act on a hidden variable. This is a *partially observed* problem (a POMDP), and no amount of training fixes it, because the information simply is not there to condition on.

Fix. Put the missing variable into the state. We added a per-room *appreciation-progress* vector to the observation, so "have I seen enough of this room?" became a function of something the policy can actually read. The lesson is the most basic one in the vocabulary section, made painful by experience: the state must contain everything the optimal action depends on, or you are not solving the MDP you think you are.

17.3 Reward shaping that backfired (reward hacking)

Symptom. Well-meant extra reward terms made behaviour worse. A "boredom" penalty meant to discourage lingering produced *looping*; a strong pull toward completing a tour produced *ping-pong* between two rooms.

Concept. Every term you add to a reward is a new objective, and the optimiser will satisfy it in whatever way scores highest, including ways you did not intend. This is *reward hacking*: circling a loop technically avoided the boredom penalty more cheaply than exiting did; an over-weighted completion pull made oscillating between two "progress" rooms score better than actually finishing. The agent was not malfunctioning; it was obeying, too literally, a reward we wrote carelessly.

Fix. Remove the boredom penalty entirely and reduce the tour pull to a gentle nudge, letting the satiation term carry "stop lingering" on its own. The lesson: prefer the *minimal* shaping that points the right way, and stress-test every term by asking "what is the cheapest way for a ruthless optimiser to collect this, and is that what I want?"

17.4 The do-nothing trap (a safe action with delayed reward)

Symptom. Given a free "decline this masterpiece" button, both visitor types learned to decline all three, even though booking was worth far more, and even after a curriculum that first showed booking paying off.

Concept. Declining is *immediately safe*, costs nothing right now, while the reward for booking is *delayed* to check-in. Faced with a safe-now option versus a better-but-later one, an optimiser that has not yet learned the delayed payoff will park on the safe action, and once parked it stops exploring the alternative, so it never discovers the payoff. This is a *do-nothing local optimum*, a close relative of the exploration problem: the agent is stuck on a hill that is locally fine and globally poor.

Fix. Here we judged the action itself unrealistic for these visitor types and removed it from the formulation, treating "always books" as a type trait (section 14.5). The general lesson is that a voluntary action that is always immediately safe but only delayed-rewarding will be abused unless either the formulation rules it out or exploration is strong enough to bridge the delay.

17.5 The self-referential runaway (feedback into your own state)

Symptom. The agent's estimate of its own arrival time drifted later and later each update, until it "planned" to arrive at closing, even on a fixed, sensible walk.

Concept. The arrival estimate was an online average of the agent's *own* past arrivals, and it fed back into the policy that produced those arrivals. A quantity that is a function of the policy's history, feeding the policy, is a feedback loop; any small drift compounds, like an alarm clock you keep resetting to whenever you happened to wake up. It also makes the environment *non-stationary* from the agent's point of view: the rules it is learning against are quietly changing because of its own behaviour.

Fix. Freeze the prior: compute the arrival estimate once from reference walks and hold it fixed during training. Breaking the loop restores a stationary target. The lesson: beware any state variable that is a function of the policy's own past and feeds back into the policy; either freeze it or model the feedback explicitly.

18 Common confusions, answered

A handful of questions come up every time these results are presented. Answering them collects the guide's ideas into a few sharp points.

Is a bigger reward number always the goal? No. The reward is a tool we *designed* to point the agent at behaviour we want, and the agent will maximise exactly what we wrote, loopholes included (the reward-hacking failure). A higher number is only "better" if the reward faithfully encodes good behaviour. Half the project was getting the reward to mean what we intended, not getting the agent to maximise it.

Why not just compute the optimal visit with a solver? Because it is not a static puzzle. The crowd is stochastic and only partially observed, the payoff for a booking is delayed by hundreds of steps, and a good policy must generalise to crowds it has never exactly seen. A shortest-path solver answers "given this fixed map, what is the quickest route?"; our question is "what should I decide now, under uncertainty about what comes next, to do well on average across many possible days?" That is a sequential decision problem, and learning is the natural tool for it.

Why does the more advanced method sometimes lose? Because "advanced" is not "better for this problem." Double Q-learning is a genuine improvement on Q-learning *when there is overestimation to remove*, and on the deterministic toy there is none, so it ties. DQN is a powerful method, but its max over noisy values is a liability on a noisy crowd, where PPO's max-free, clipped updates are simply better suited. The competent move is to match the method to the structure of the problem, and our results are a case study in what happens when you do and do not.

Why average five seeds rather than report the best run? Because the best run is cherry-picking, and it can hide fragility. The art-lover/max-crowd cell looks like a clean +41% on a lucky seed, but across five it is revealed as a coin-flip between a large gain and a collapse. The mean with its standard deviation is the honest summary: it states both the typical outcome and how much we can trust it.

If we hand the agent the legal moves, is it really learning? Yes, it is learning the part that matters. Masking removes only the trivial sub-problem of which moves physically exist; the agent still has to learn *when* to book, *how far* ahead, and how to pace a long visit so every window is met before closing. We deliberately spend its capacity on those genuine decisions rather than on rediscovering the floor plan.

19 Glossary

State s
Everything the agent knows when it must act; a complete-enough snapshot.

Action a
A choice available in a state; the legal set can depend on the state.

Reward r
The immediate score the world returns after one action.

Return G
The (discounted) sum of all rewards from now to the end; what we maximise.

Discount γ
Weight on future rewards; near 1 is far-sighted, near 0 short-sighted.

Policy π
The strategy: a map from states to actions (or action probabilities).

Value $V(s)$, $Q(s, a)$
Expected return from a state, or from a state-action pair.

MDP / Markov
The formal problem (states, actions, rewards, transitions, discount); "Markov" means the state summarises the past well enough that the future depends only on it.

TD error
The gap between a freshly bootstrapped estimate and the old one; the thing every value method shrinks.

Bootstrapping
Updating an estimate toward a target that itself contains an estimate.

On-policy / off-policy
Learning about the policy you are running (PPO) versus about the optimal one while behaving otherwise (Q-learning, DQN).

Overestimation
The upward bias from taking a max over noisy estimates; cured by Double-Q / Double-DQN.

Function approximation
Using a neural network instead of a table to compute values or probabilities, so similar states share learning.

Gradient descent
Nudging the network's weights in the direction that most reduces a loss.

Advantage $\hat{A}_t$
How much better an action turned out than the critic expected; the clean signal PPO learns from.

GAE
Generalised advantage estimation; blends short- and long-horizon advantage estimates via λ.

Clipping
PPO's brake that forbids the policy ratio from moving more than $\pm\epsilon$ per update.

Entropy bonus
A small reward for keeping action probabilities spread out, to sustain exploration.

Action masking
Forcing illegal actions to zero probability before the softmax.

Replay buffer
DQN's memory of past transitions, replayed in random batches.

Target network
A slowly updated copy of the Q-network that supplies stable targets.

Deadly triad
Bootstrapping + function approximation + off-policy learning; together, a recipe for instability.

Common random numbers (CRN)
Scoring every policy on the same fixed crowd scenarios so differences reflect the policy, not luck.

Seed
The random-number starting point of a run; we average over five to get error bars.

Credit assignment
The problem of attributing a delayed outcome to the earlier action that caused it.

Fill curve
The rule tying slot availability to lead time and crowd: busier days need longer leads.

RAMA
Reservation-Anchored Masterpiece Access; the booked-slot system the agent learns to use.